



Code optimisation

on intel xeon (co)Processors

PARALLELIZATION STRATEGIES



* Partitioning

- * Break computation into small pieces

* Communication

- * Identify communication among pieces

* Agglomeration

- * Group pieces to avoid communication

* Mapping

- * Map the pieces to the computational devices

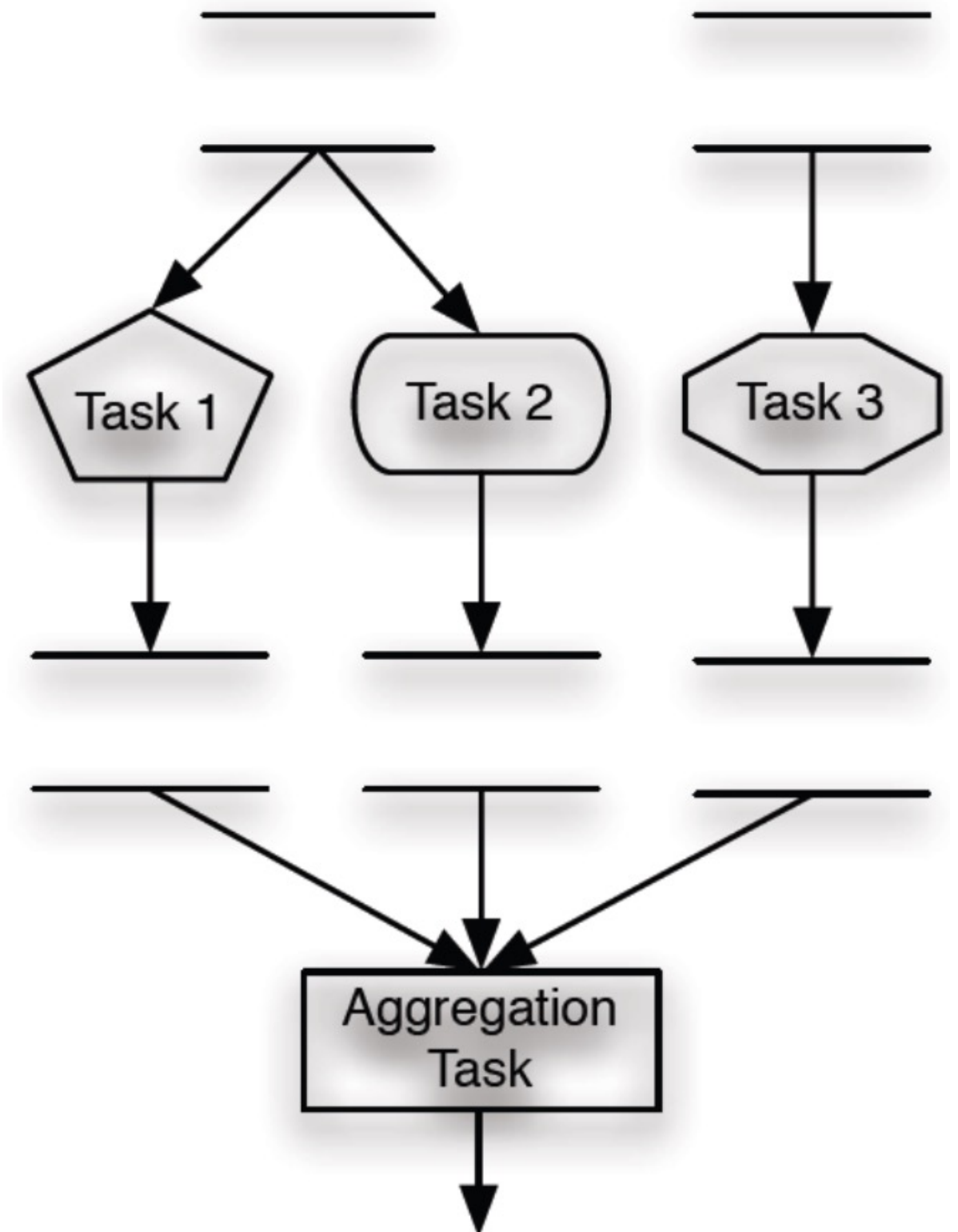


- ✱ Task parallelism
- ✱ Divide-and-conquer decomposition
- ✱ Geometric decomposition
- ✱ Recursive data decomposition
- ✱ Pipeline decomposition

Task Parallelism



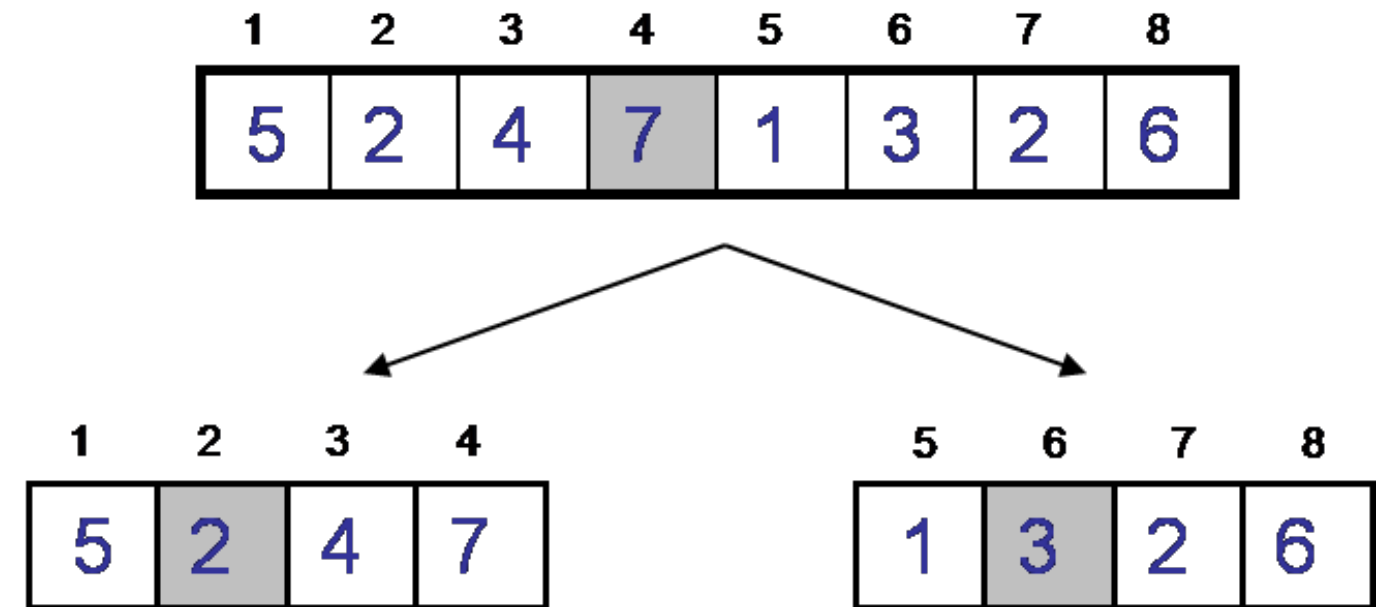
- ✱ Divide the workload into tasks
- ✱ Process independent tasks in parallel
- ✱ Map tasks to processes/threads
- ✱ Aggregate the results



Divide-and-Conquer



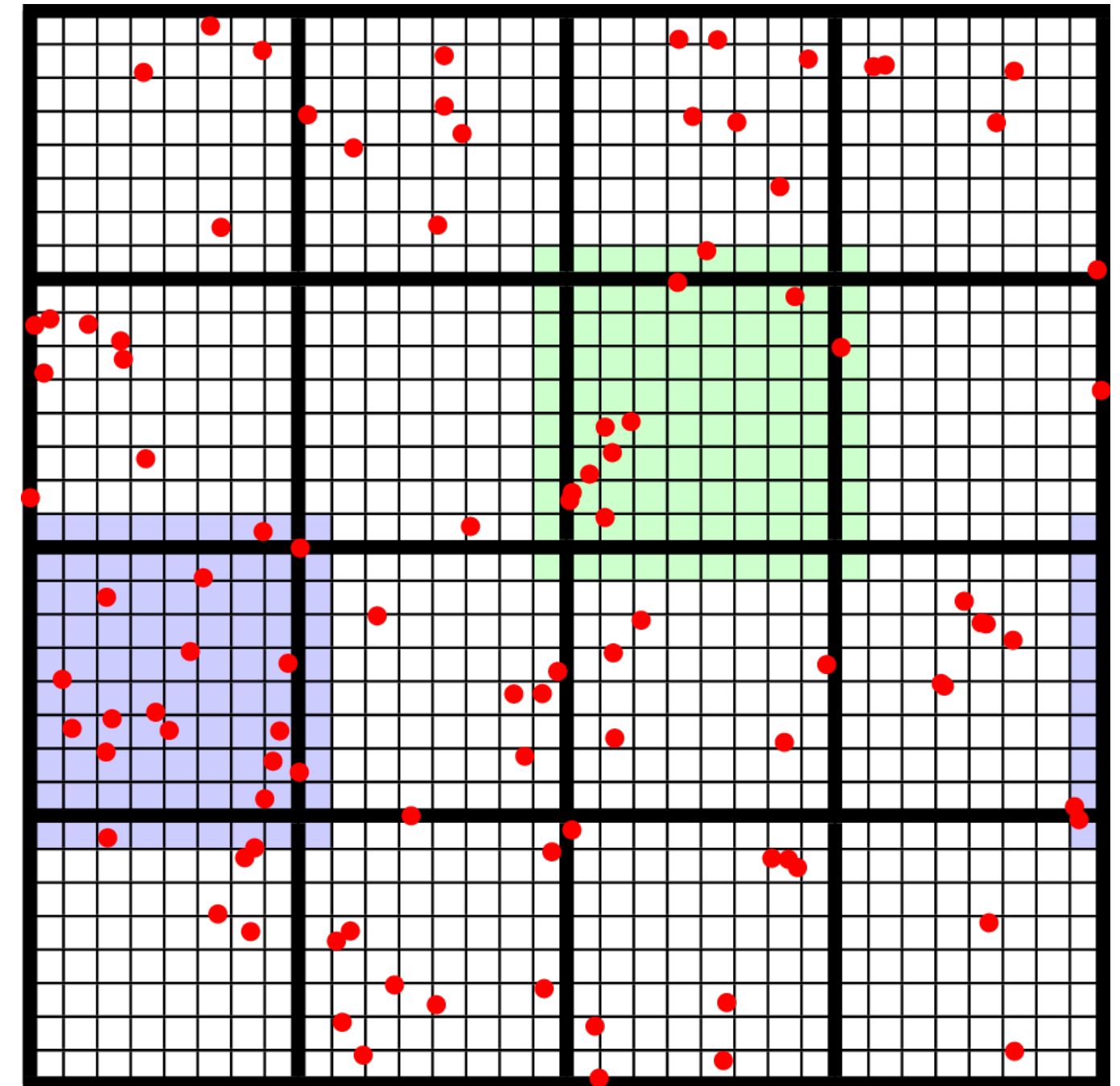
- ✱ Divide the algorithm into tasks
- ✱ Tasks can be subdivided
- ✱ Map groups of tasks to processes/threads



Geometric Decomposition



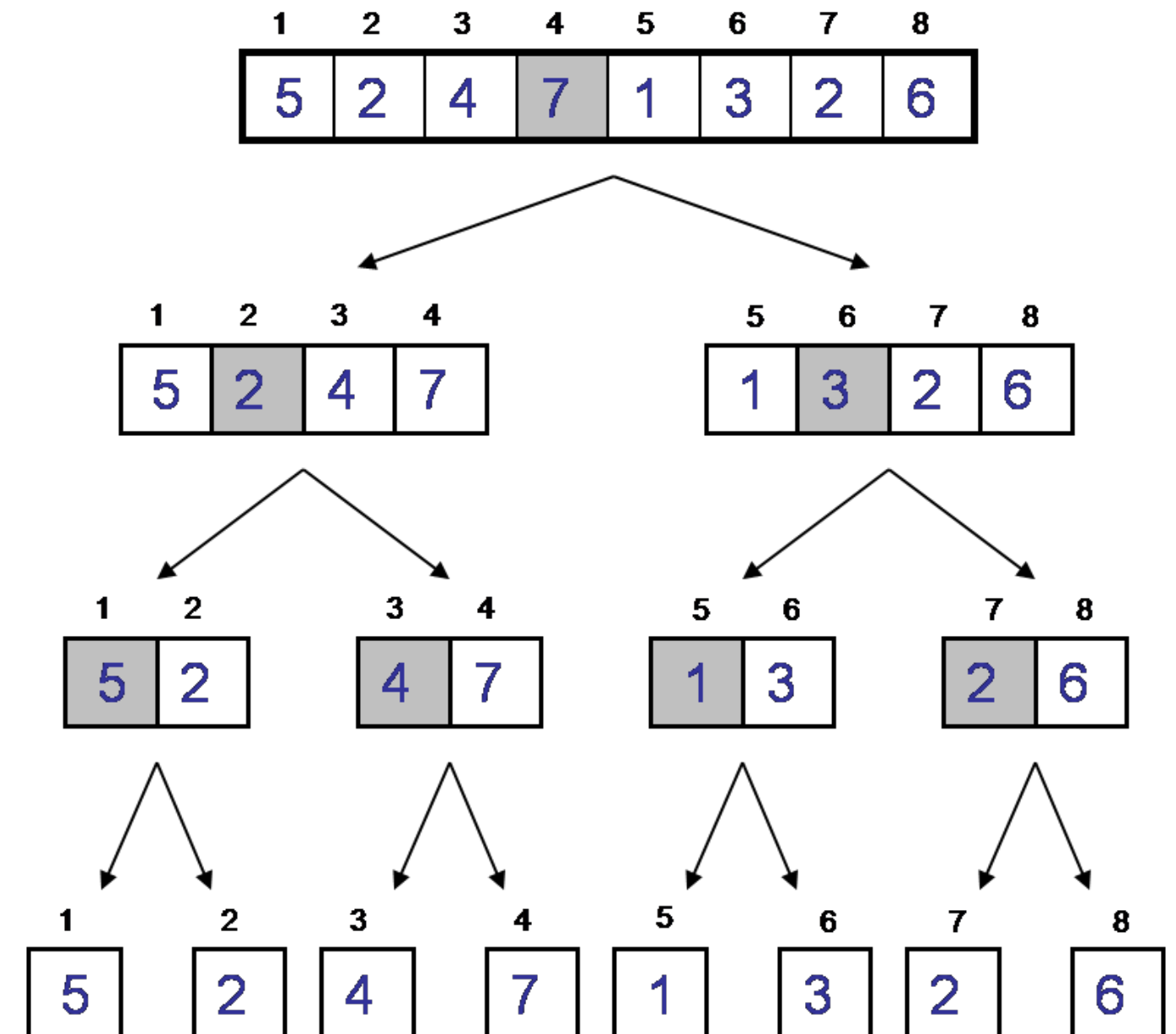
- ✱ Divide the geometric space
- ✱ Each sub-space is assigned to a process/thread
- ✱ Communication may be required in the boundary regions



Recursive Decomposition



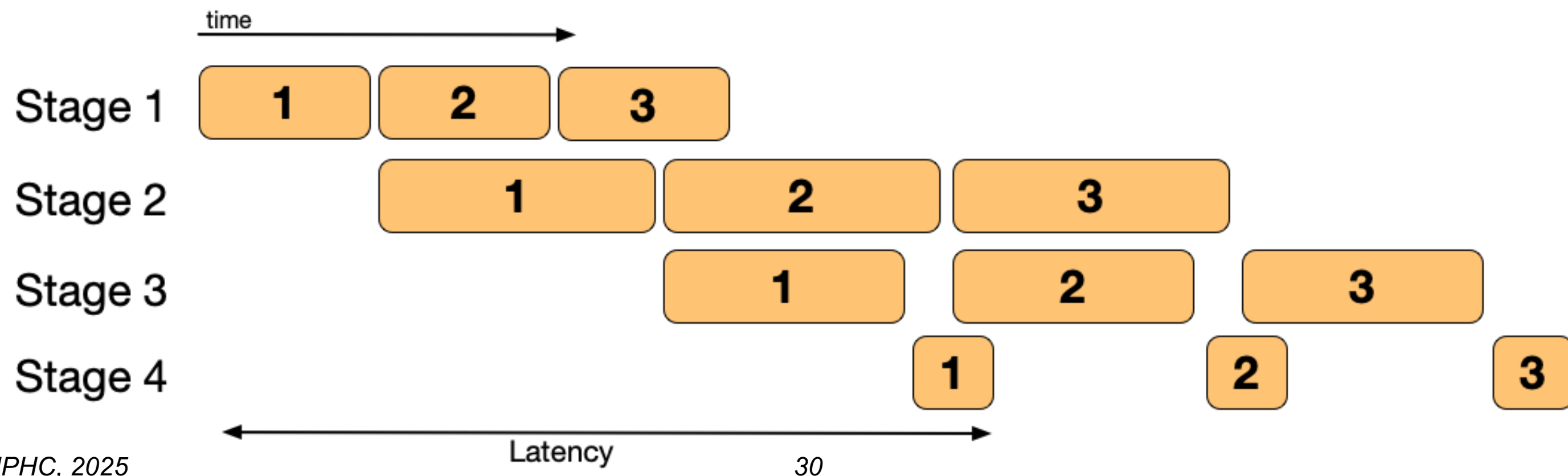
- * Data structures that cannot be easily partitioned
 - * Trees, lists, graphs, ...
- * Tasks **must** be subdivided recursively
- * Process the tree of tasks from the bottom
- * Map groups of tasks to processes/threads



Pipeline Decomposition



- ✱ Sets of subsequent tasks (pipeline stages) applied to independent data
- ✱ Data can be processed simultaneously by different pipeline stages
- ✱ Assumes that the same stage cannot process multiple items simultaneously

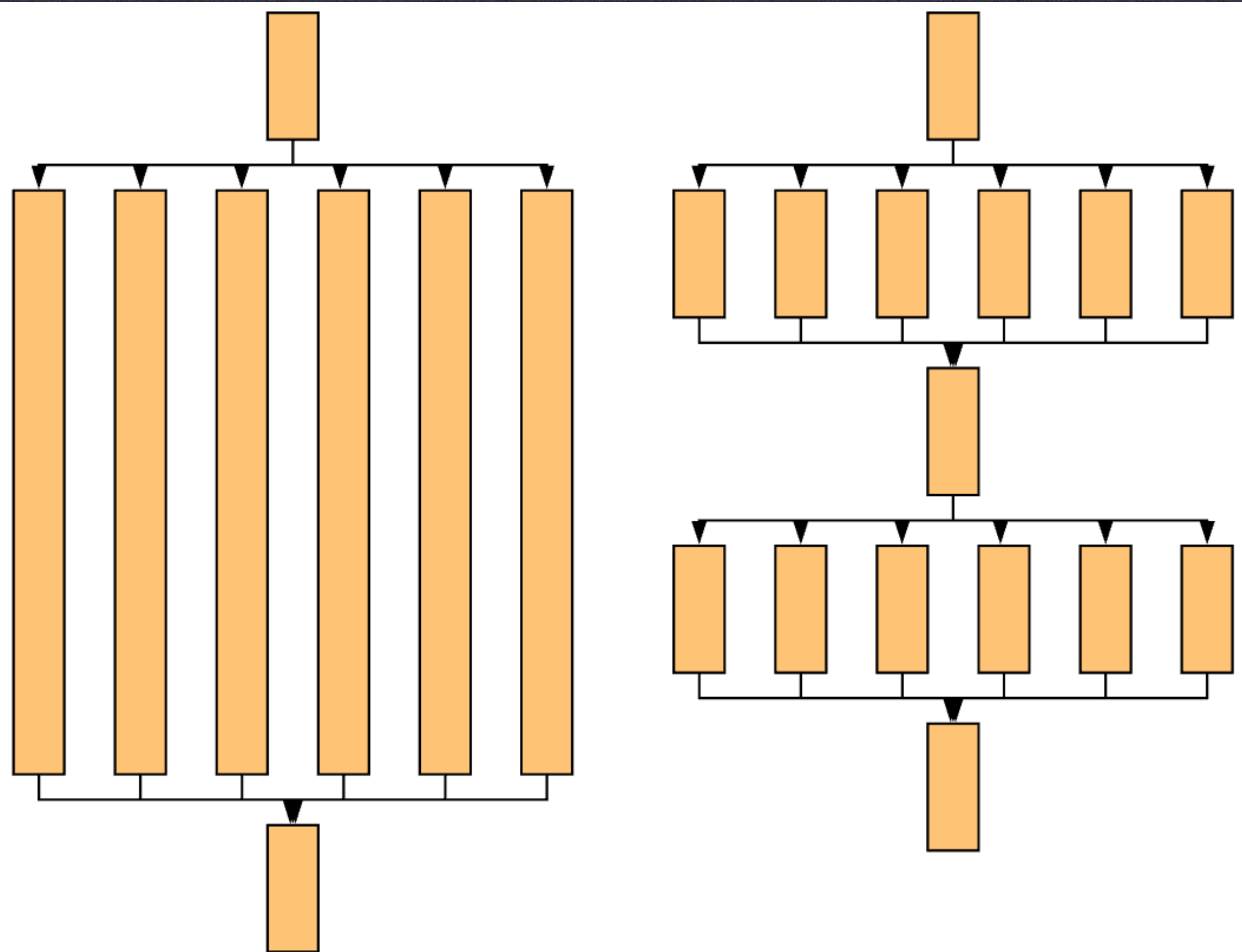


PARALLELIZATION IMPLEMENTATION



- ✱ **Globally parallel, locally sequential** - multiple tasks performed simultaneously, each task sequential
 - ✱ Single program, multiple data (SPMD)
 - ✱ Multiple program, multiple data (MPMD)
 - ✱ Master-worker
 - ✱ Map-reduce
- ✱ **Globally sequential, locally parallel** - sequential application with individual parallel sections
 - ✱ Fork-join
 - ✱ Loop parallelism

GPLS VS GSLP





✱ SPMD

- ✱ Single executable
- ✱ All devices compute the same code on different data

✱ MPMD

- ✱ Different executables for each device
- ✱ Useful for heterogeneous nodes (x86 + ARM, PU + GPU)

✱ Master-worker

- ✱ Separates management and processing roles
- ✱ Single master handles work to multiple workers using a strategy

✱ Map-reduce

- ✱ Maps independent data to different workers

- ✱ Reduces (merges) the results of each worker³⁴



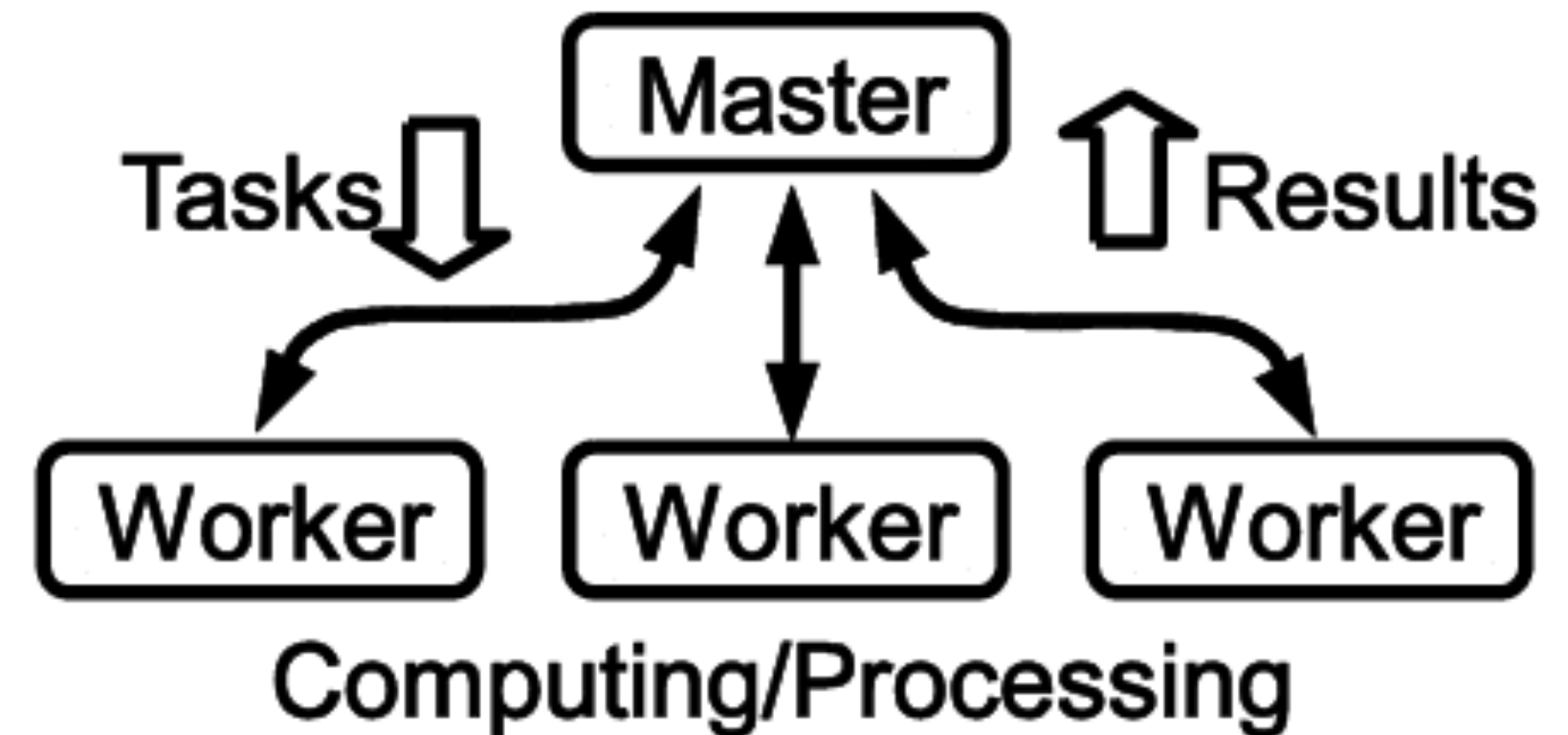
- ✱ Single code base is easier to maintain
- ✱ Typical structure
 - ✱ Initialisation - thread pool and management
 - ✱ Obtain unique thread identifier - scalar or vector id
 - ✱ Run the code according to each id - workload distribution
 - ✱ Shutdown - thread shutdown and result merge
- ✱ Prone to algorithm logic errors due to data races



- ✱ Multiple code bases - harder to maintain
- ✱ May require multiple different programming paradigms
- ✱ Different device architectures adds another layer of irregularity
- ✱ Should only be adopted due to apps RAM memory requirements or to use accelerators
- ✱ Can be used in conjunction with SPMD strategies



- ✱ Clear definition of tasks for masters and workers
- ✱ Masters
 - ✱ Hand out work to workers - how to distribute it?
 - ✱ Collect results from workers
 - ✱ Interacts with files and with the users





- ✱ A derivative of master-worker
 - ✱ Popularised by the initial releases of the Google search engine
- ✱ Multiple workers spawned to run the same code
 - ✱ Can share intermediate results among them
 - ✱ Results are merged by the master at the end
- ✱ Master-worker usually implies a set of different tasks and/or different executables
 - ✱ Map-reduce uses a single code base in a single executable



✱ Fork-join

- ✱ Creation of dynamic tasks (processes/threads) that must complete for the parent to continue
- ✱ Multiple instances of this behaviour per application

✱ Loop parallelism

- ✱ Subset of fork-join
- ✱ Creates multiple processes/threads with the same code to process different blocks of independent data



- ✱ Used when the algorithm requires dynamic creation of tasks at runtime
 - ✱ Parent task must wait for children tasks to finish to continue
 - ✱ Tasks can be processed by pools of pre-generated processes/threads
- ✱ Can lead to excessive amounts of idle times if not implemented properly



- ✱ The most simple and convenient method for most problems
 - ✱ Loop iterations are divided into chunks
 - ✱ Chunks are assigned to computing threads
 - ✱ Static and dynamic strategies to assign chunks are often available in libraries - which should be used?
- ✱ Is not efficient in distributed environments

Match Decomposition to Implementation



	Task Parallelism	Divide-Conquer	Geometric	Recursive	Pipeline
SPMD	✓	✓	✓	✓	✓
MPMD	✓	✓	✓	✓	✓
Master-Worker	✓	✓	✓	✓	
Map-Reduce		✓	✓	✓	
Fork-Join	✓	✓	✓	✓	✓
Parallel Loop		✓	✓		



- ✱ An Overview of Programming for Intel® Xeon® processors and Intel® Xeon Phi™ coprocessors, James Reinders, Intel
- ✱ Intel® Xeon Phi™ Coprocessor High Performance Programming, Jim Jeffers, James Reinders, Elsevier Waltham (Mass.), 2013
- ✱ Intel® 64 and IA-32 Architectures Software Developer's Manual